

DEBER 9 - P3

MiniTienda - Registro y análisis de ventas

Documento de evidencias del desarrollo

El programa implementa una MiniTienda por consola para mantener un catálogo de productos, registrar ventas, trabajar con archivos CSV, calcular métricas, generar gráficos y controlar la interacción mediante un menú. El desarrollo utiliza tuplas, listas, diccionarios, funciones, Pandas, NumPy y Matplotlib.

Requisito	Cumplimiento
Tuplas, listas y diccionarios	Cumplido
Funciones y modularización	Cumplido
Pandas: DataFrame, groupby y CSV	Cumplido
NumPy: mean, std y sum	Cumplido
Matplotlib: gráfica de barras	Cumplido
ventas.csv con mínimo 10 ventas	Cumplido
log.txt	Cumplido
try/except/else/finally	Cumplido
while, for, if/elif/else, break, continue	Cumplido
Retos A, B, C y D	Implementados en el desarrollo

Explicación breve del algoritmo

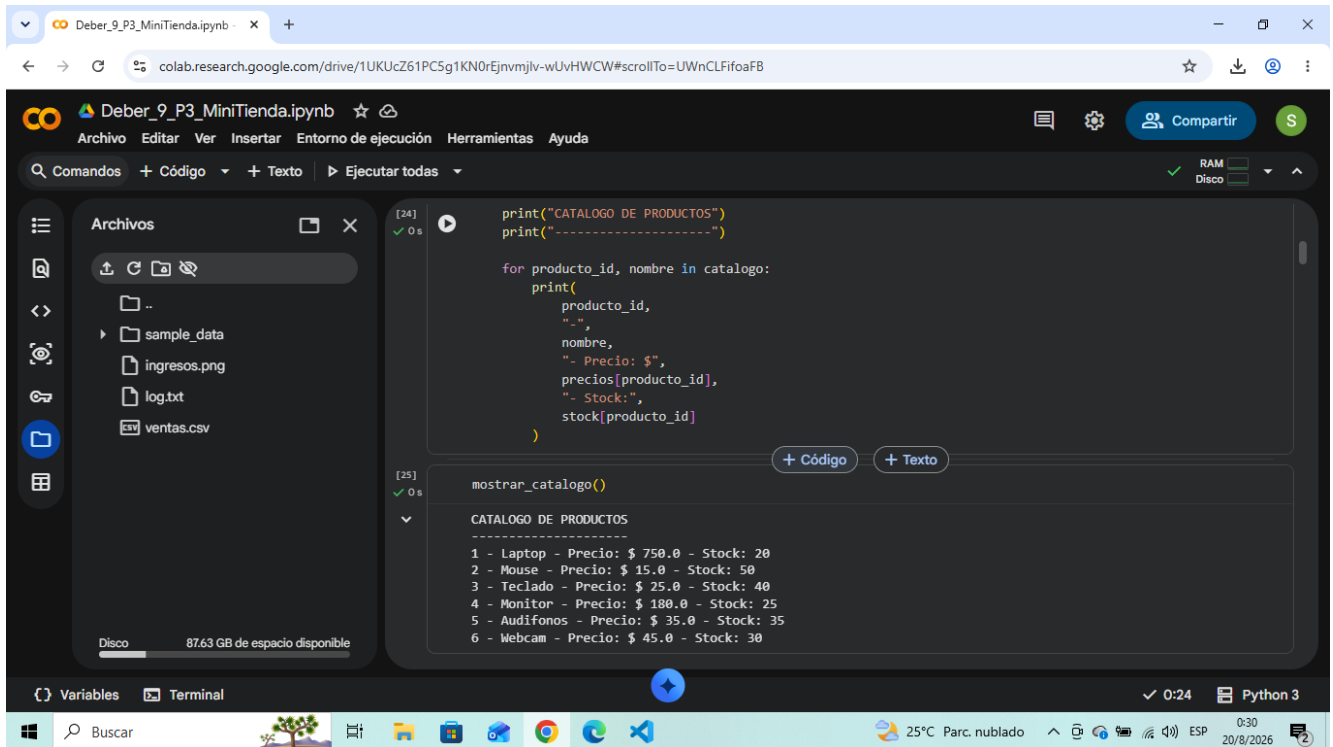
Primero se define el catálogo y la información de precios y stock. La función de registro solicita el producto y la cantidad, valida los datos, comprueba la existencia y disponibilidad del producto, calcula el subtotal y aplica un descuento del 5% cuando se compran 10 o más unidades. Después registra la venta y actualiza el stock.

Las ventas se transforman en un DataFrame de Pandas y se guardan en ventas.csv. Pandas también agrupa los ingresos por producto. NumPy toma los totales para obtener promedio, desviación estándar y suma. Matplotlib utiliza los ingresos agrupados para crear una gráfica de barras y permite exportarla como ingresos.png.

El menú principal funciona con un ciclo while. Según la opción seleccionada se ejecuta una función diferente. Se utilizan if/elif/else para las decisiones, for para recorrer datos, continue para regresar al menú cuando una operación no puede continuar y break para finalizar el programa. Los bloques try/except controlan entradas no numéricas, archivos inexistentes y división por cero.

Catálogo de productos

Se muestra el catálogo de la MiniTienda. Los productos están almacenados mediante tuplas, mientras que los precios y el stock se manejan con diccionarios.



The screenshot shows a Google Colab notebook titled "Deber_9_P3_MiniTienda.ipynb". The notebook contains two code cells. The first cell (line 24) defines a tuple for products, a dictionary for prices, and a dictionary for stock. The second cell (line 25) calls a function `mostrar\_catalogo()` which prints the catalog. The output of the second cell shows the following product catalog:

```
print("CATALOGO DE PRODUCTOS")
print("-----")

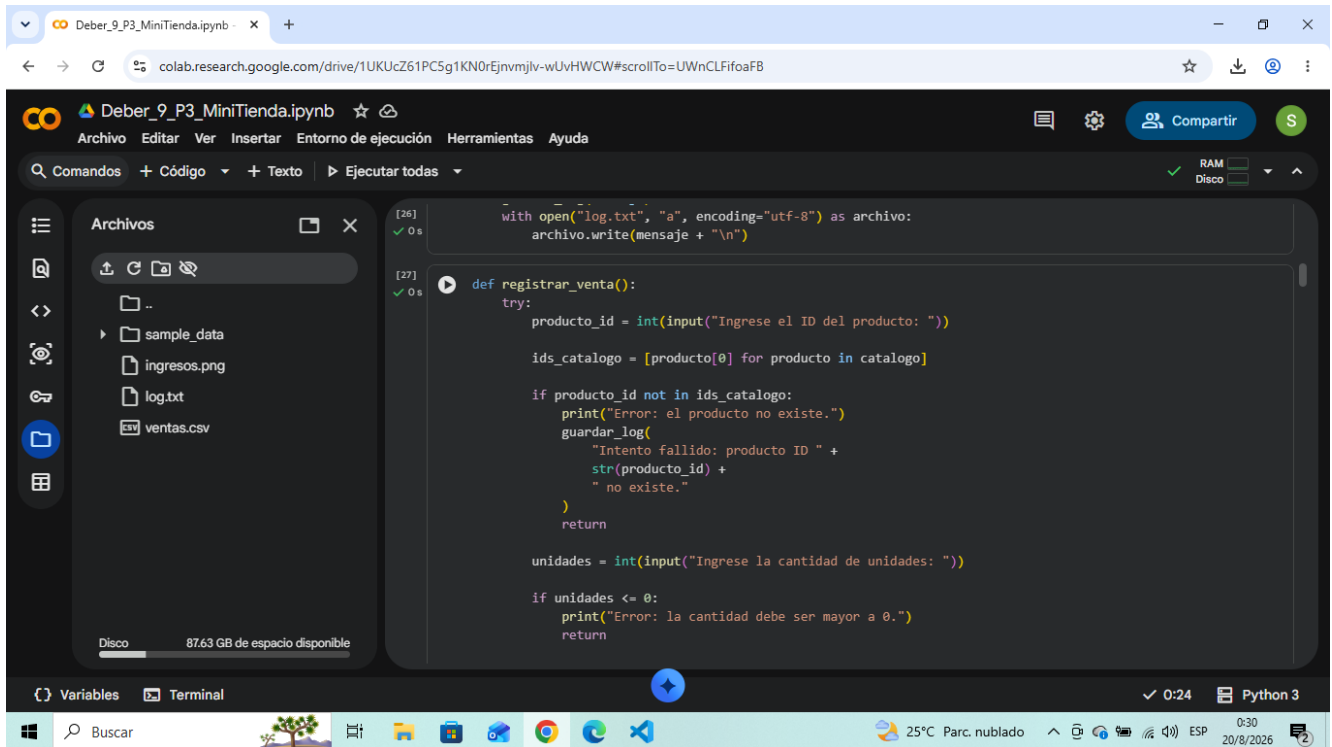
for producto_id, nombre in catalogo:
    print(
        producto_id,
        "\n",
        nombre,
        "- Precio: $",
        precios[producto_id],
        "- Stock:",
        stock[producto_id]
    )
```

```
mostrar_catalogo()

CATALOGO DE PRODUCTOS
-----
1 - Laptop - Precio: $ 750.0 - Stock: 20
2 - Mouse - Precio: $ 15.0 - Stock: 50
3 - Teclado - Precio: $ 25.0 - Stock: 40
4 - Monitor - Precio: $ 180.0 - Stock: 25
5 - Audifonos - Precio: $ 35.0 - Stock: 35
6 - Webcam - Precio: $ 45.0 - Stock: 30
```

Registro de ventas y validación de producto

La función registrar_venta() solicita el ID y las unidades. También verifica que el producto exista y registra en log.txt los intentos fallidos.



The screenshot shows a Google Colab notebook titled "Deber_9_P3_MiniTienda.ipynb". The left sidebar displays a file explorer with the following files: sample_data, ingresos.png, log.txt, and ventas.csv. The main editor area contains Python code for the registrar_venta() function. The code includes a try-except block for product validation and a log file update. The bottom status bar indicates the notebook is running on Python 3.

```
[26] ✓ 0 s with open("log.txt", "a", encoding="utf-8") as archivo:
      archivo.write(mensaje + "\n")

[27] ✓ 0 s def registrar_venta():
      try:
          producto_id = int(input("Ingrese el ID del producto: "))

          ids_catalogo = [producto[0] for producto in catalogo]

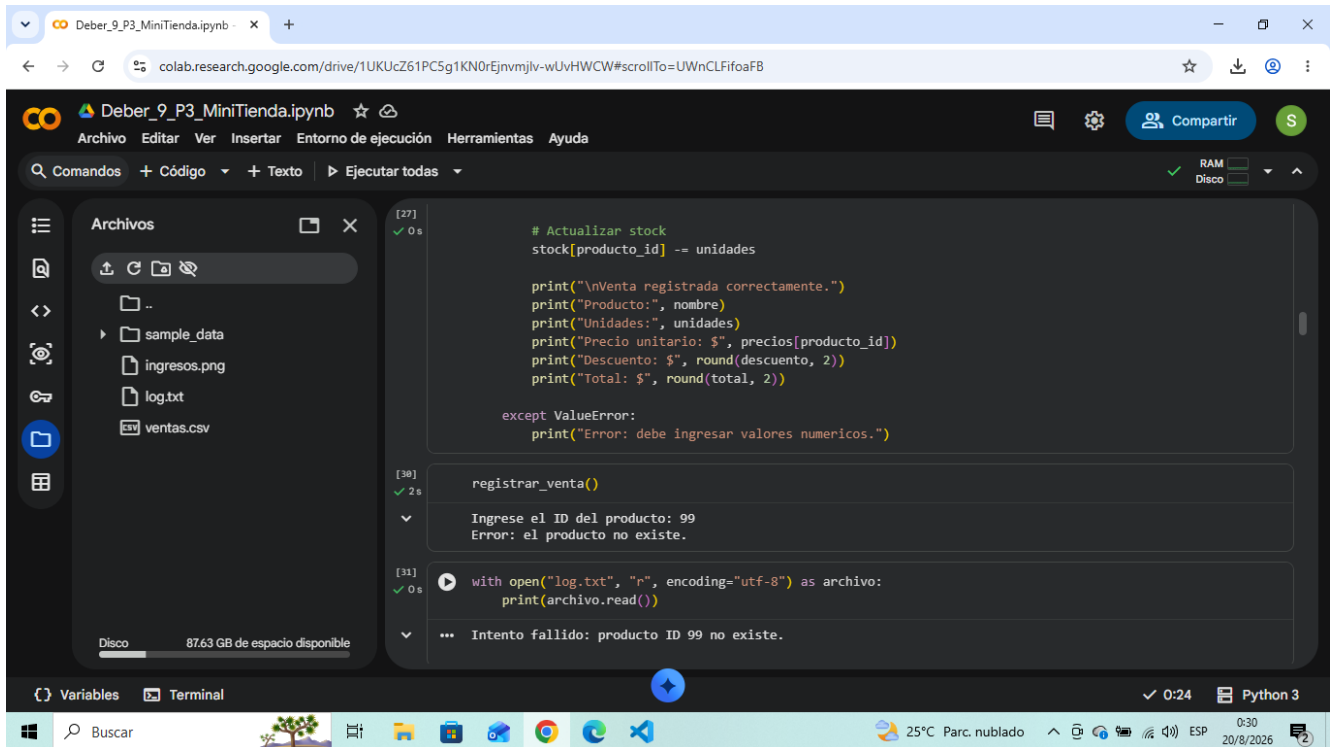
          if producto_id not in ids_catalogo:
              print("Error: el producto no existe.")
              guardar_log(
                  "Intento fallido: producto ID " +
                  str(producto_id) +
                  " no existe."
              )
              return

          unidades = int(input("Ingrese la cantidad de unidades: "))

          if unidades <= 0:
              print("Error: la cantidad debe ser mayor a 0.")
              return
```

Reto D - Registro de intento fallido

Se realizó una prueba con el producto ID 99. El programa detectó que no existe y guardó el intento fallido en log.txt.



The screenshot shows a Google Colab notebook titled "Deber_9_P3_MiniTienda.ipynb". The left sidebar displays a file explorer with a folder named "sample_data" and files named "ingresos.png", "log.txt", and "ventas.csv". The main editor area contains Python code for a mini-store system. The code includes a function to update stock, a function to register a sale, and a main loop that prompts the user for product ID and units. The execution output shows the following steps:

- Line 27: Code execution successful (0 s).
- Line 30: Code execution successful (2 s).
- Line 31: Code execution successful (0 s).

The output of the program shows the following messages:

```
# Actualizar stock
stock[producto_id] -= unidades

print("\nVenta registrada correctamente.")
print("Producto:", nombre)
print("Unidades:", unidades)
print("Precio unitario: $", precios[producto_id])
print("Descuento: $", round(descuento, 2))
print("Total: $", round(total, 2))

except ValueError:
    print("Error: debe ingresar valores numericos.")

registrar_venta()

Ingrese el ID del producto: 99
Error: el producto no existe.

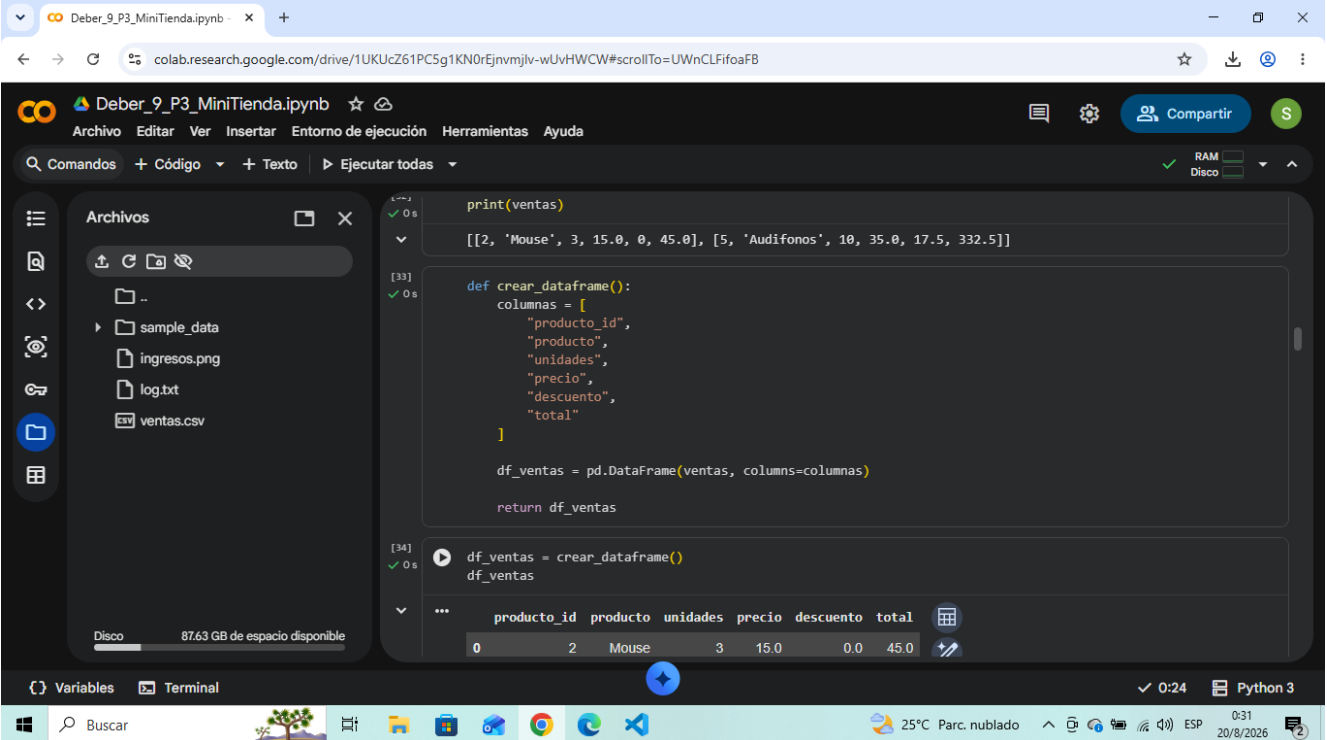
with open("log.txt", "r", encoding="utf-8") as archivo:
    print(archivo.read())

... Intento fallido: producto ID 99 no existe.
```

The bottom status bar indicates the notebook is running on Python 3, with a memory usage of 0:24 and a date of 20/8/2026.

Pandas - Creación del DataFrame

Las ventas almacenadas en listas se convierten en un DataFrame de Pandas con las columnas producto_id, producto, unidades, precio, descuento y total.



The screenshot shows a Google Colab notebook titled "Deber_9_P3_MiniTienda.ipynb". The notebook is open to a cell containing the following Python code:

```
print(ventas)

[[2, 'Mouse', 3, 15.0, 0, 45.0], [5, 'Audifonos', 10, 35.0, 17.5, 332.5]]

def crear_dataframe():
    columnas = [
        "producto_id",
        "producto",
        "unidades",
        "precio",
        "descuento",
        "total"
    ]

    df_ventas = pd.DataFrame(ventas, columns=columnas)

    return df_ventas

df_ventas = crear_dataframe()
df_ventas
```

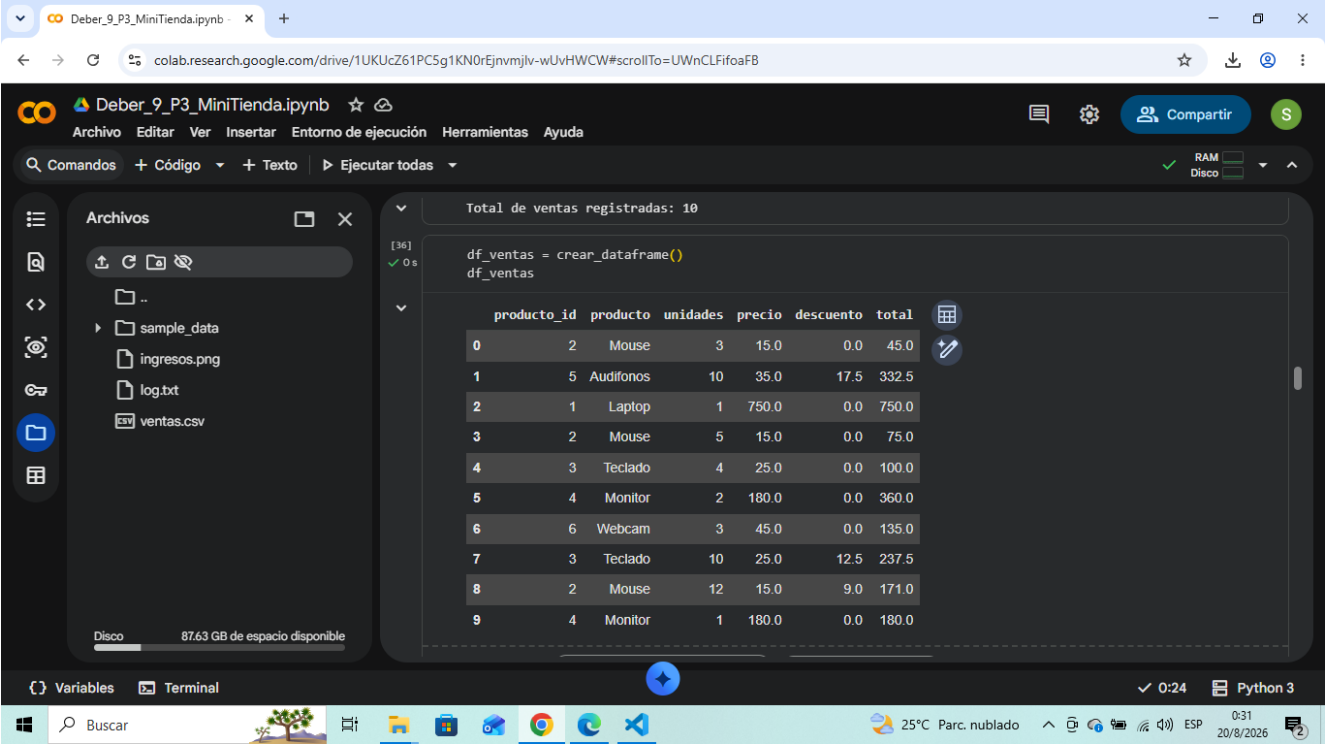
The output of the code is a Pandas DataFrame with the following columns: producto_id, producto, unidades, precio, descuento, and total. The DataFrame contains two rows of data:

producto_id	producto	unidades	precio	descuento	total
2	Mouse	3	15.0	0.0	45.0
5	Audifonos	10	35.0	17.5	332.5

The notebook interface also shows a file explorer on the left with files like sample_data, ingresos.png, log.txt, and ventas.csv. The bottom status bar indicates the notebook is running on Python 3.

Diez ventas registradas

Se comprueba el requisito de contar con al menos 10 ventas para generar y analizar el archivo ventas.csv.



The screenshot shows a Google Colab notebook titled "Deber_9_P3_MiniTienda.ipynb". The notebook is open to a cell containing the following code:

```
[36]  
✓ 0 s  
df_ventas = crear_dataframe()  
df_ventas
```

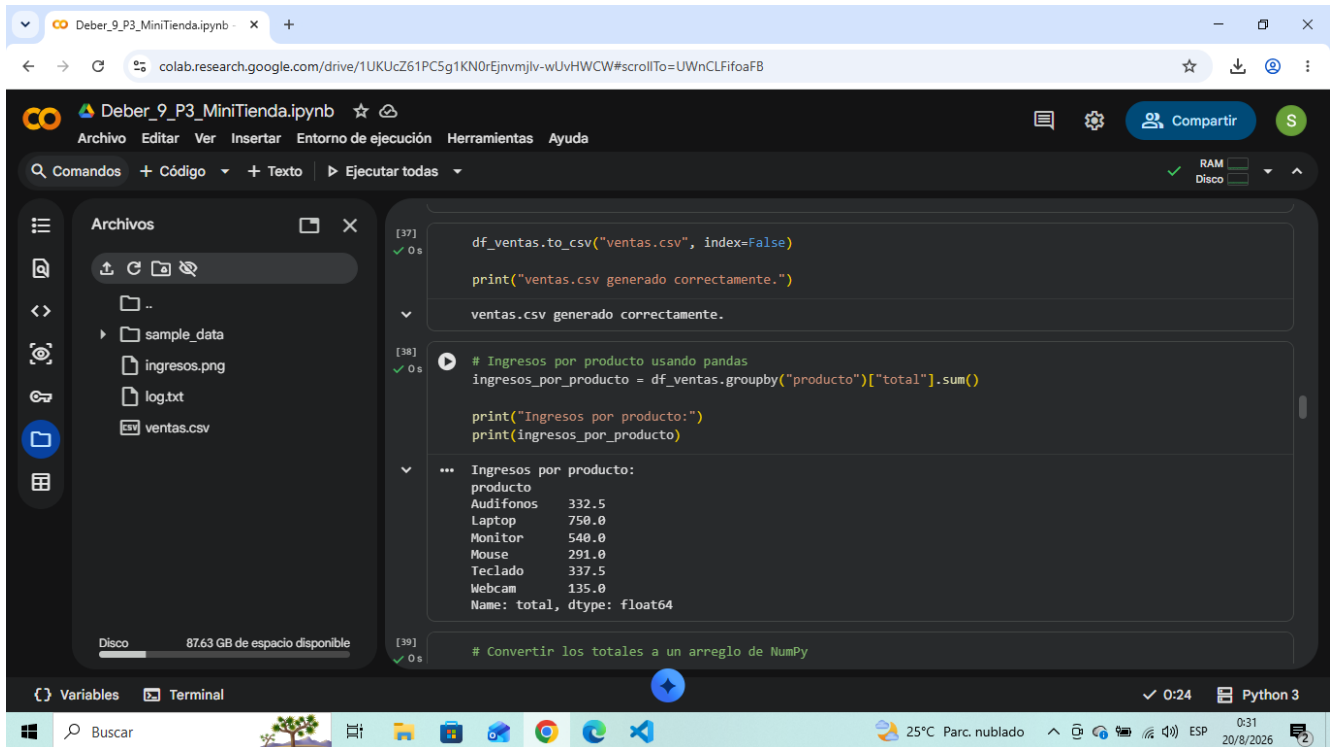
The output of the code is a DataFrame with 10 rows of sales data. The columns are: producto_id, producto, unidades, precio, descuento, and total. The data is as follows:

	producto_id	producto	unidades	precio	descuento	total
0	2	Mouse	3	15.0	0.0	45.0
1	5	Audifonos	10	35.0	17.5	332.5
2	1	Laptop	1	750.0	0.0	750.0
3	2	Mouse	5	15.0	0.0	75.0
4	3	Teclado	4	25.0	0.0	100.0
5	4	Monitor	2	180.0	0.0	360.0
6	6	Webcam	3	45.0	0.0	135.0
7	3	Teclado	10	25.0	12.5	237.5
8	2	Mouse	12	15.0	9.0	171.0
9	4	Monitor	1	180.0	0.0	180.0

The notebook interface also shows a file explorer on the left with files like sample_data, ingresos.png, log.txt, and ventas.csv. The bottom status bar indicates the system is at 25°C, with a cloudy sky, and the date is 20/8/2026.

CSV y agrupación con Pandas

El DataFrame se guarda en ventas.csv. Con groupby() se agrupan las ventas por producto y se calcula el ingreso total correspondiente a cada uno.



The screenshot shows a Google Colab notebook titled "Deber_9_P3_MiniTienda.ipynb". The left sidebar displays a file explorer with a folder named "sample_data" containing files "ingresos.png", "log.txt", and "ventas.csv". The main code area shows the following Python code:

```
[37] df_ventas.to_csv("ventas.csv", index=False)
      print("ventas.csv generado correctamente.")

ventas.csv generado correctamente.

[38] # Ingresos por producto usando pandas
      ingresos_por_producto = df_ventas.groupby("producto")["total"].sum()

      print("Ingresos por producto:")
      print(ingresos_por_producto)

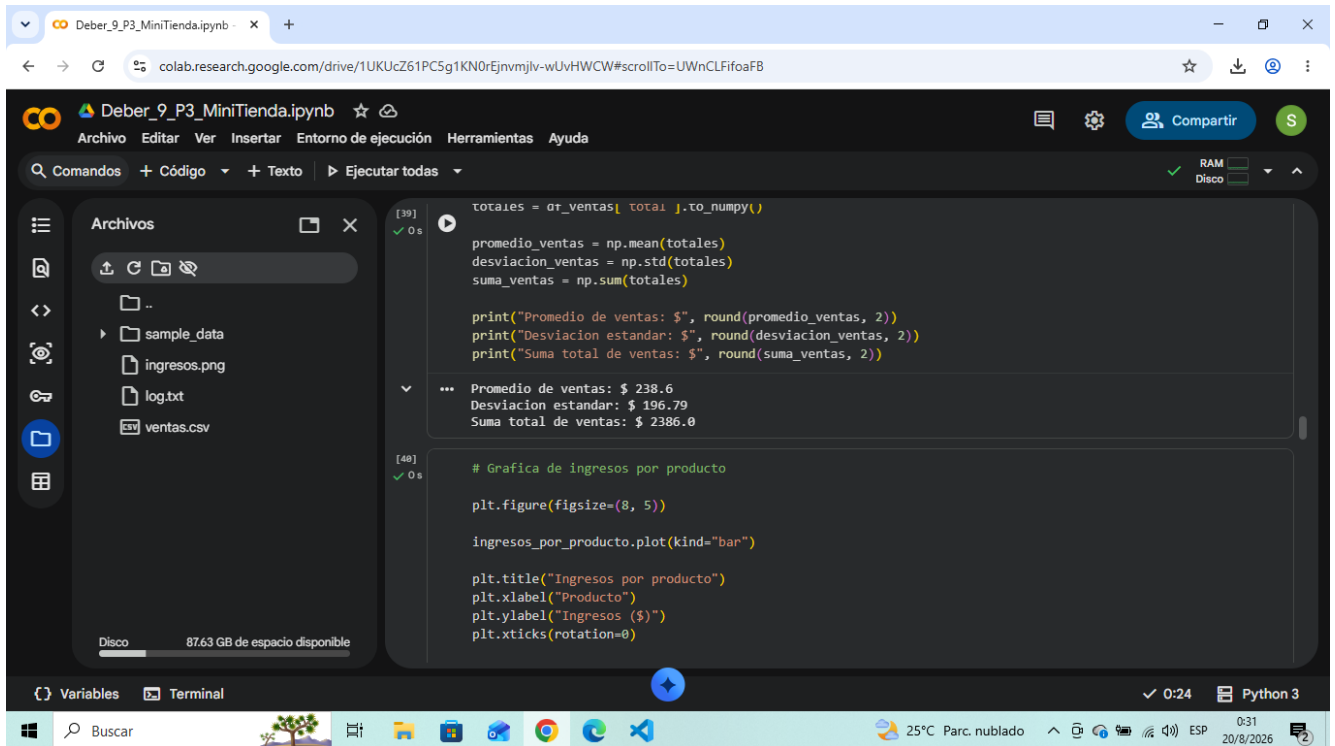
... Ingresos por producto:
      producto
Audifonos    332.5
Laptop       758.0
Monitor      548.0
Mouse        291.0
Teclado      337.5
Webcam       135.0
Name: total, dtype: float64

[39] # Convertir los totales a un arreglo de NumPy
```

The bottom of the notebook shows a status bar with "Variables", "Terminal", and "Python 3". The system tray at the very bottom indicates a temperature of 25°C, a cloudy sky, and the date 20/8/2026.

Métricas con NumPy

NumPy se utiliza para calcular el promedio, la desviación estándar y la suma total de los valores de venta.



The screenshot shows a Google Colab notebook titled "Deber_9_P3_MiniTienda.ipynb". The left sidebar displays a file explorer with a folder named "sample_data" containing "ingresos.png", "log.txt", and "ventas.csv". The main area shows two code cells. The first cell, labeled [39], calculates the mean, standard deviation, and sum of sales using NumPy. The second cell, labeled [48], creates a bar chart titled "Ingresos por producto". The output of the first cell shows the following values:

```
[39]
✓ 0 s

totales = df_ventas[ 'total' ].to_numpy()

promedio_ventas = np.mean(totales)
desviacion_ventas = np.std(totales)
suma_ventas = np.sum(totales)

print("Promedio de ventas: $", round(promedio_ventas, 2))
print("Desviacion estandar: $", round(desviacion_ventas, 2))
print("Suma total de ventas: $", round(suma_ventas, 2))

... Promedio de ventas: $ 238.6
Desviacion estandar: $ 196.79
Suma total de ventas: $ 2386.0
```

The output of the second cell shows the following code:

```
[48]
✓ 0 s

# Grafica de ingresos por producto

plt.figure(figsize=(8, 5))

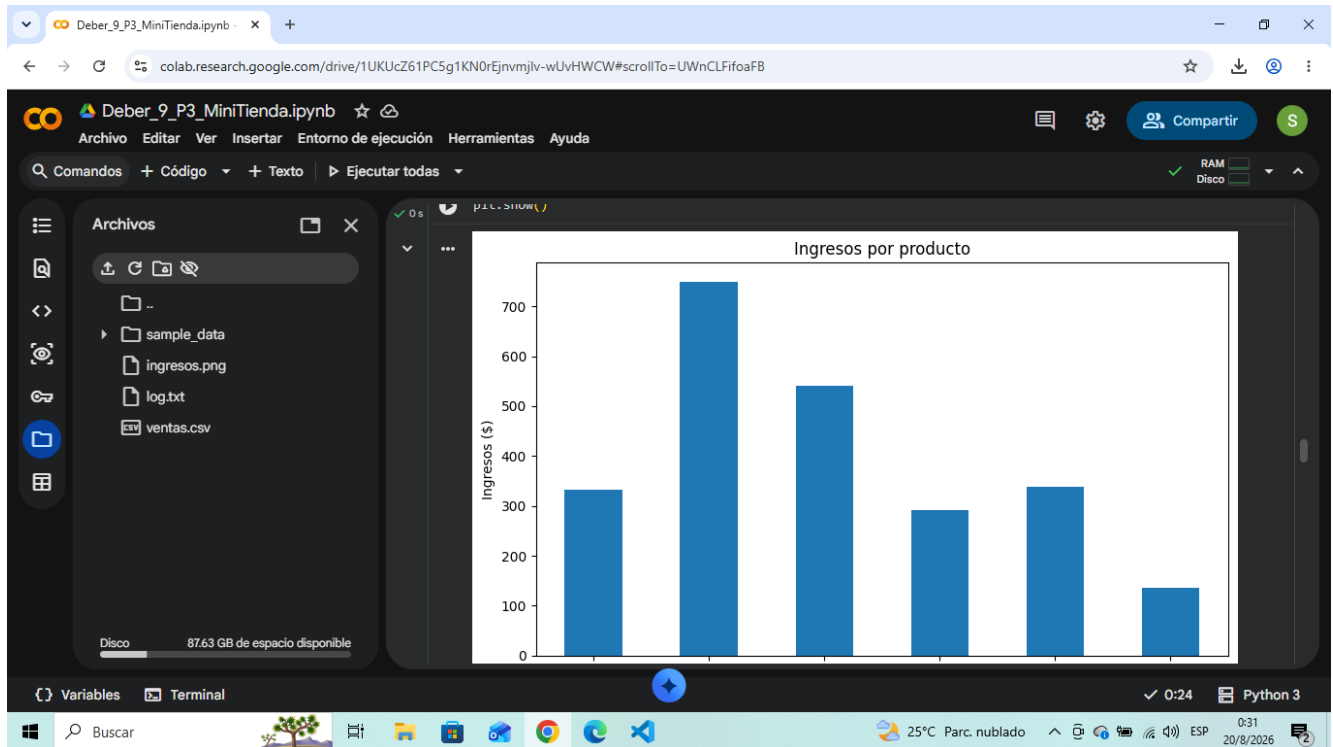
ingresos_por_producto.plot(kind="bar")

plt.title("Ingresos por producto")
plt.xlabel("Producto")
plt.ylabel("Ingresos ($)")
plt.xticks(rotation=0)
```

The bottom status bar indicates the notebook is running on Python 3, with a temperature of 25°C and a date of 20/8/2026.

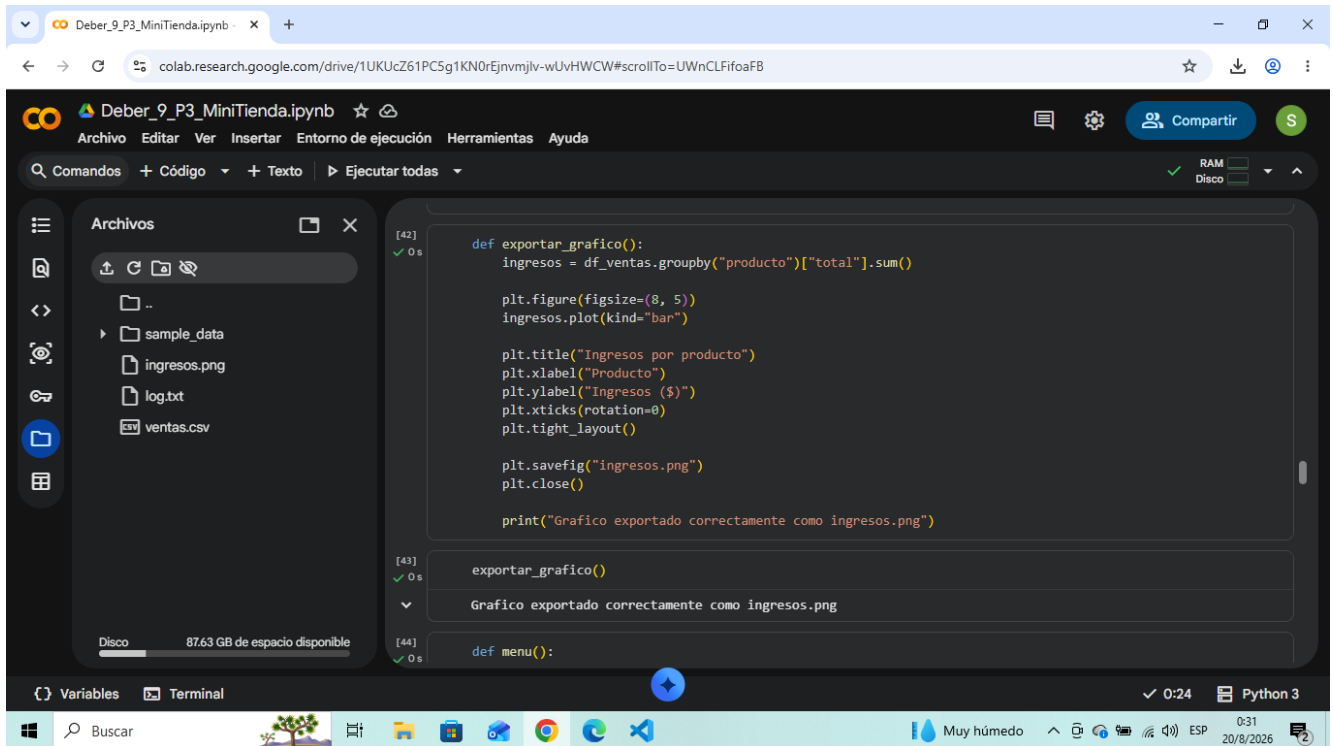
Gráfico de ingresos por producto

Matplotlib genera una gráfica de barras que permite comparar visualmente los ingresos obtenidos por cada producto.



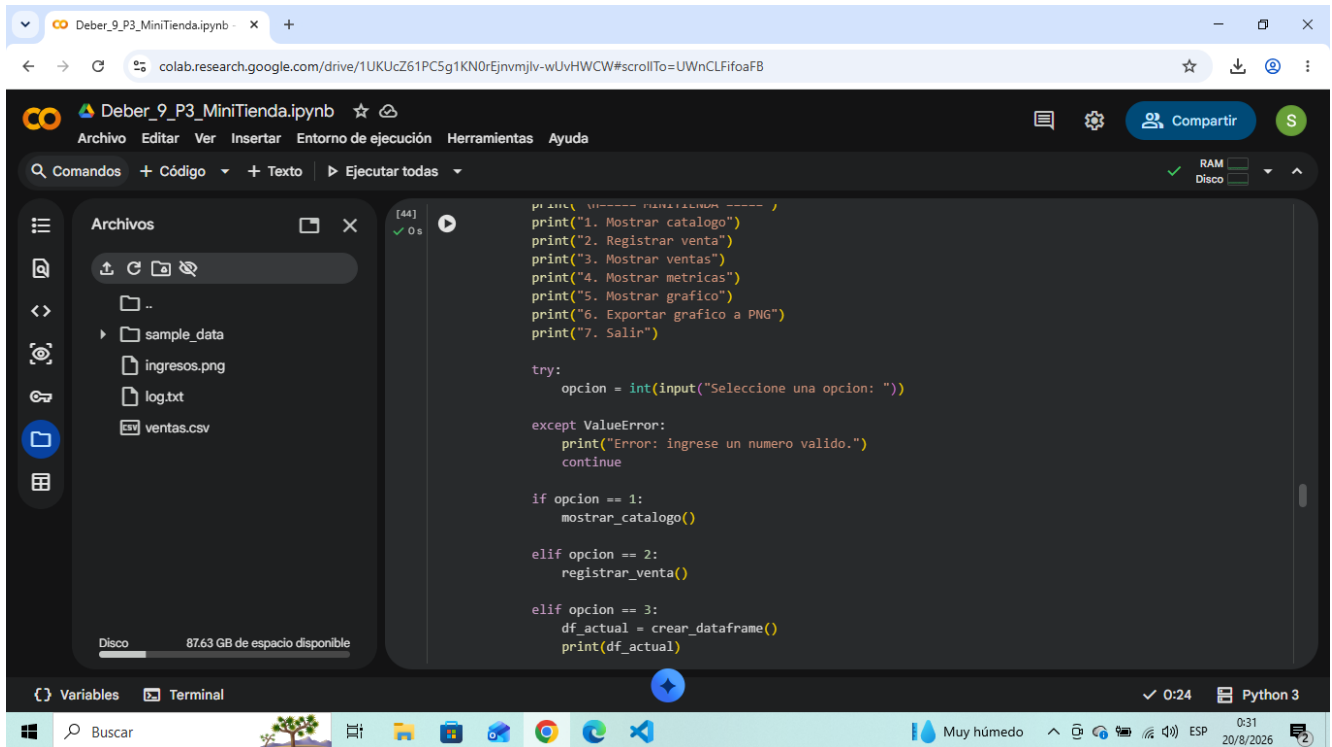
Reto B - Exportación del gráfico

La función `exportar_grafico()` utiliza `plt.savefig('ingresos.png')` para guardar la gráfica en formato PNG.



Menú principal y control de flujo

El programa incluye un menú que trabaja dentro de un ciclo while. Se utilizan try/except, if/elif/else, continue y las distintas opciones solicitadas.



```
print("----- MENÚ TIENDA -----")
print("1. Mostrar catalogo")
print("2. Registrar venta")
print("3. Mostrar ventas")
print("4. Mostrar metricas")
print("5. Mostrar grafico")
print("6. Exportar grafico a PNG")
print("7. Salir")

try:
    opcion = int(input("Seleccione una opcion: "))
except ValueError:
    print("Error: ingrese un numero valido.")
    continue

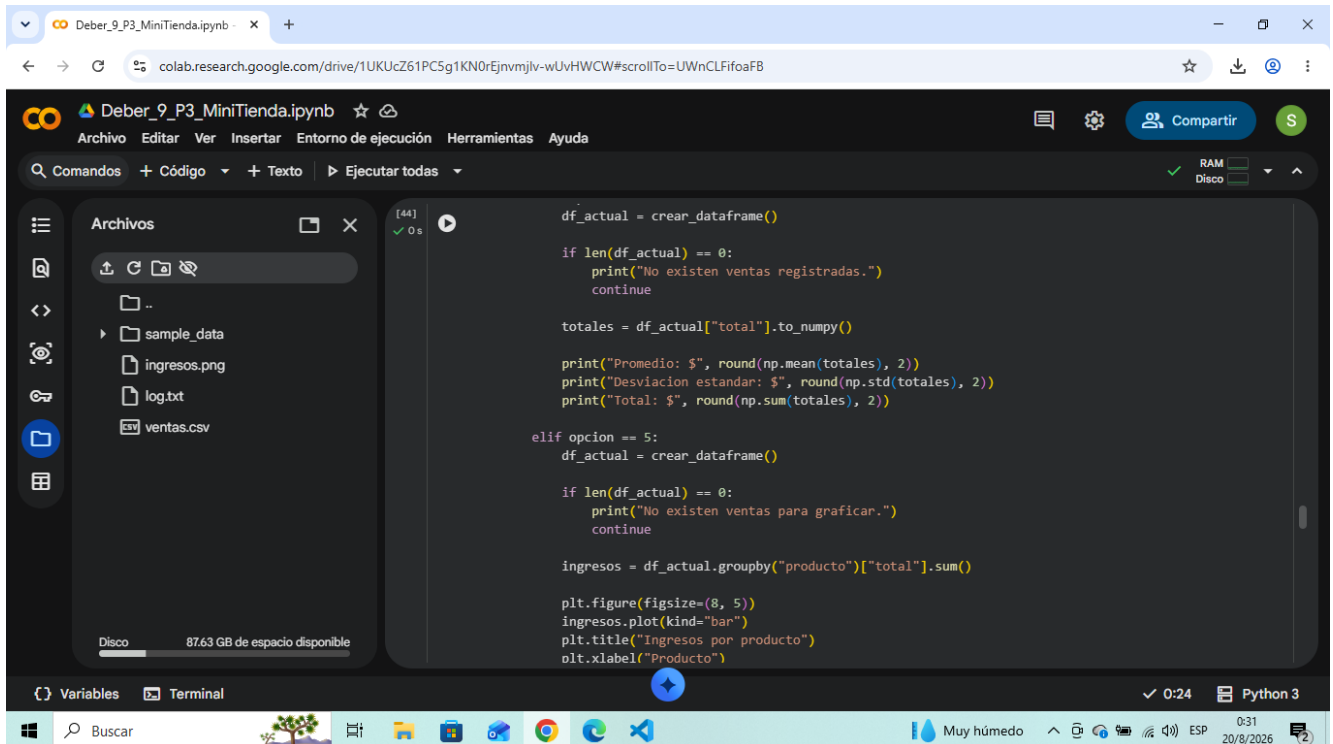
if opcion == 1:
    mostrar_catalogo()

elif opcion == 2:
    registrar_venta()

elif opcion == 3:
    df_actual = crear_dataframe()
    print(df_actual)
```

Control del menú

Las opciones permiten mostrar métricas, graficar, exportar el gráfico y salir mediante break.



```
df_actual = crear_dataframe()

if len(df_actual) == 0:
    print("No existen ventas registradas.")
    continue

totales = df_actual["total"].to_numpy()

print("Promedio: $", round(np.mean(totales), 2))
print("Desviacion estandar: $", round(np.std(totales), 2))
print("Total: $", round(np.sum(totales), 2))

elif opcion == 5:
    df_actual = crear_dataframe()

    if len(df_actual) == 0:
        print("No existen ventas para graficar.")
        continue

    ingresos = df_actual.groupby("producto")["total"].sum()

    plt.figure(figsize=(8, 5))
    ingresos.plot(kind="bar")
    plt.title("Ingresos por producto")
    plt.xlabel("Producto")
```

Lectura de ventas.csv

La lectura del archivo utiliza try/except para controlar FileNotFoundError. También se incluyen else y finally para completar el manejo de excepciones.

```
[44] ✓ 0 s
plt.xticks(rotation=8)
plt.tight_layout()
plt.show()

elif opcion == 6:
    exportar_grafico()

elif opcion == 7:
    print("Programa finalizado.")
    break

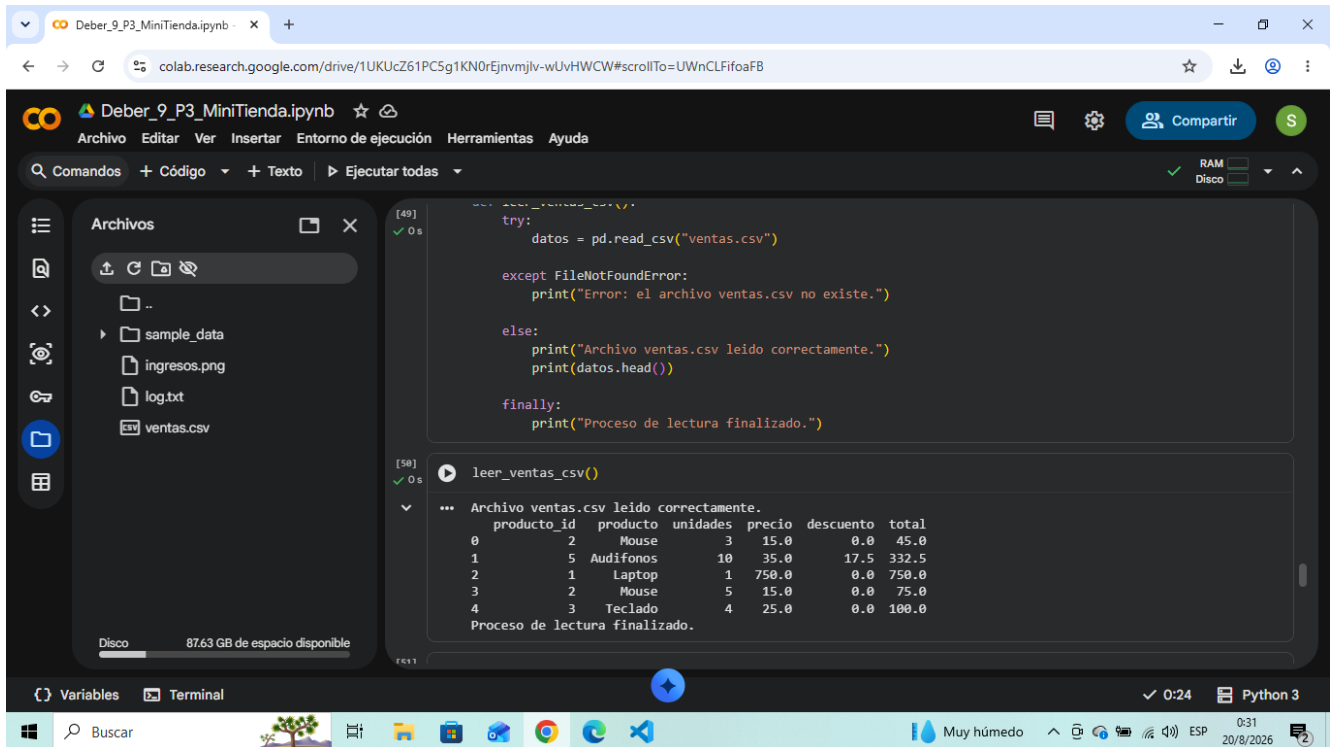
else:
    print("Opcion no valida.")

[48] 0 s
menu()

===== MINITIENDA =====
1. Mostrar catalogo
2. Registrar venta
3. Mostrar ventas
4. Mostrar metricas
5. Mostrar grafico
6. Exportar grafico a PNG
7. Salir
```

Control de división por cero

Se demuestra el control de ZeroDivisionError mediante try/except. El bloque finally se ejecuta al finalizar el cálculo.



The screenshot shows a Google Colab notebook titled "Deber_9_P3_MiniTienda.ipynb". The left sidebar displays a file explorer with a folder named "sample_data" containing files "ingresos.png", "log.txt", and "ventas.csv". The main code cell (cell 49) contains a try/except/finally block that attempts to read the "ventas.csv" file using pandas. The output of the code cell shows the successful reading of the CSV file and the first five rows of data.

```
[49] ✓ 0 s
try:
    datos = pd.read_csv("ventas.csv")

except FileNotFoundError:
    print("Error: el archivo ventas.csv no existe.")

else:
    print("Archivo ventas.csv leído correctamente.")
    print(datos.head())

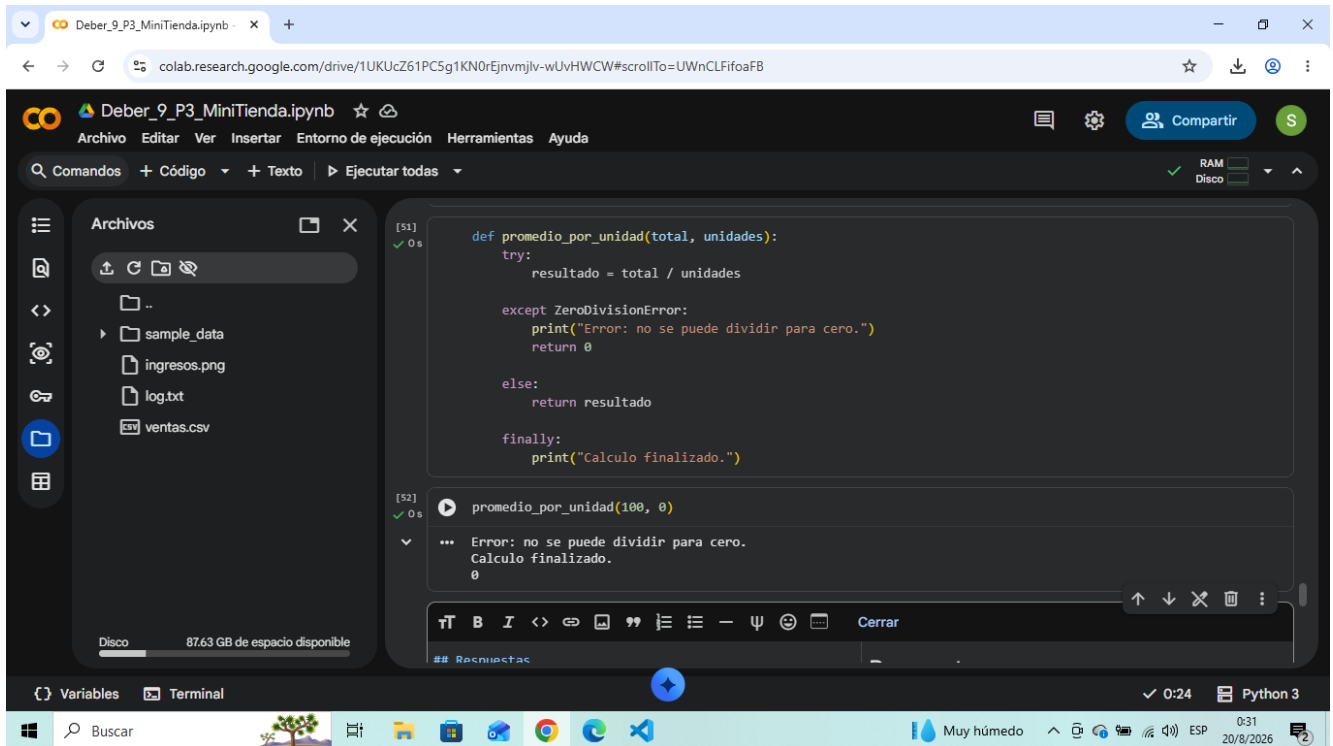
finally:
    print("Proceso de lectura finalizado.")
```

```
[50] ✓ 0 s
leer_ventas_csv()

... Archivo ventas.csv leído correctamente.
  producto_id  producto  unidades  precio  descuento  total
0           2      Mouse          3    15.0         0.0   45.0
1           5  Audifonos         10    35.0        17.5  332.5
2           1    Laptop          1   750.0         0.0  750.0
3           2      Mouse          5    15.0         0.0   75.0
4           3   Teclado          4    25.0         0.0  100.0
Proceso de lectura finalizado.
```

Respuestas de la actividad

Se explica el uso de Pandas y NumPy y se describe dónde se aplicó try/except dentro del programa.



The screenshot shows a Google Colab notebook titled "Deber_9_P3_MiniTienda.ipynb". The left sidebar displays a file explorer with a folder named "sample_data" and files "ingresos.png", "log.txt", and "ventas.csv". The main editor area contains a Python function definition and its execution output.

```
[51] ✓ 0 s
def promedio_por_unidad(total, unidades):
    try:
        resultado = total / unidades

    except ZeroDivisionError:
        print("Error: no se puede dividir para cero.")
        return 0

    else:
        return resultado

    finally:
        print("Calculo finalizado.")
```

The function is then called with the following code:

```
[52] ✓ 0 s
promedio_por_unidad(100, 0)
```

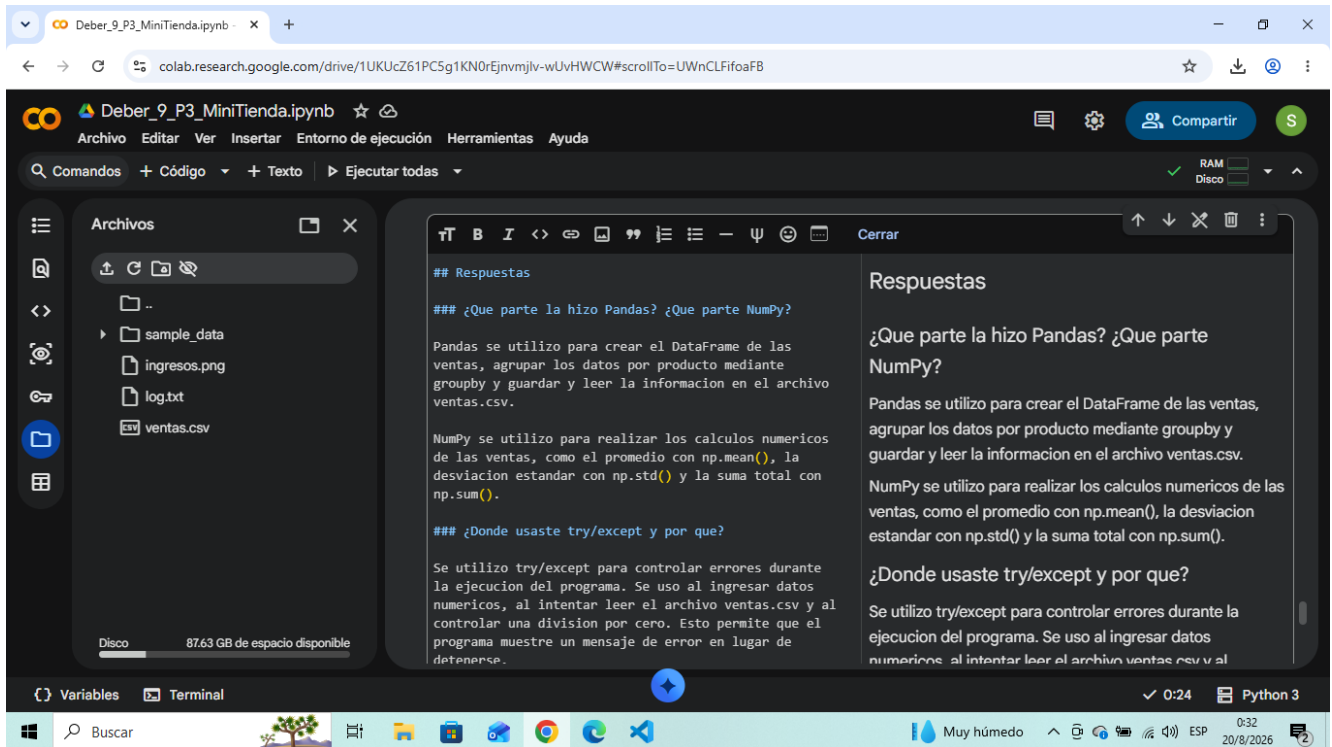
The output of the execution is:

```
... Error: no se puede dividir para cero.
Calculo finalizado.
0
```

The bottom status bar indicates the environment is Python 3 and the time is 0:24. The system tray at the bottom shows the date 20/8/2026 and the time 0:31.

Estructuras utilizadas

Se identifican las estructuras principales: catálogo como tupla, ventas como lista y precios y stock como diccionarios.



```
## Respuestas

### ¿Que parte la hizo Pandas? ¿Que parte NumPy?

Pandas se utilizo para crear el DataFrame de las
ventas, agrupar los datos por producto mediante
groupby y guardar y leer la informacion en el archivo
ventas.csv.

NumPy se utilizo para realizar los calculos numericos
de las ventas, como el promedio con np.mean(), la
desviacion estandar con np.std() y la suma total con
np.sum().

### ¿Donde usaste try/except y por que?

Se utilizo try/except para controlar errores durante
la ejecucion del programa. Se uso al ingresar datos
numericos, al intentar leer el archivo ventas.csv y al
controlar una division por cero. Esto permite que el
programa muestre un mensaje de error en lugar de
detenerse.
```

Archivos

- sample_data
- ingresos.png
- log.txt
- ventas.csv

Disco 87.63 GB de espacio disponible

Variables Terminal

Muy húmedo 0:32 20/8/2026

Respuestas

¿Qué parte la hizo Pandas? ¿Qué parte NumPy?

Pandas se utilizó para crear el DataFrame de las ventas, agrupar los datos por producto mediante groupby y guardar y leer la información en ventas.csv. NumPy se utilizó para los cálculos numéricos: promedio con np.mean(), desviación estándar con np.std() y suma total con np.sum().

¿Dónde usaste try/except y por qué?

Se utilizó try/except para controlar errores durante la ejecución, especialmente al ingresar datos numéricos, leer ventas.csv y controlar una división por cero. Esto evita que el programa se detenga por una entrada o situación incorrecta.

¿Qué estructuras son tuplas, listas y diccionarios en el código?

catalogo es una tupla que contiene los productos. ventas es una lista donde se almacenan las ventas registradas. precios y stock son diccionarios que relacionan el ID de cada producto con su precio y cantidad disponible.

Conclusión

La MiniTienda integra las estructuras y herramientas solicitadas para registrar, almacenar y analizar ventas. El programa valida los datos, mantiene el stock, registra errores, genera archivos, calcula métricas y presenta los ingresos mediante una gráfica, cumpliendo con el objetivo general de la actividad.